Keep Going Until — While Loops

A **loop** repeats a block of code, so you type one `circle()` instead of ten. Today's loop keeps going **until** something becomes true. Keep this beside you while you build; leave page 2 face-down until after.

The four parts of a while loop

ITERATION

```
let x = 220;              // 1 start
while (x <= 580) {       // 2 the question
  circle(x, 268, 16);     // 3 the body
  x = x + 40;             // 4 the update
}
```

PART	ITS JOB
start	Sets x **before** the loop. Never inside the braces.
question	A Boolean, like Unit 2. **True:** run the body again. **False:** skip past the loop.
body	The work you want repeated.
update	Changes x so the question will **eventually** be false.

Trace it

COUNT THE CHECKS

X	X <= 580?	BULB
220	true	1
…	…	…
580	true	10
620	false	stop

ONE MORE QUESTION THAN BODY
The question was asked 11 times; the body ran 10. The loop only ends because one check came back false — and that check costs a turn.

A while loop does not animate

ONE FRAME

The whole loop runs **inside one frame**. p5 only shows the canvas when `draw()` ends, and `draw()` cannot end until the loop has. You never see bulb 1, then bulb 2.

`draw()` repeats over **time**. `while` repeats **right now**.

Infinite loops

SAVE FIRST

THE TAB FREEZES
No update line (or one that moves the wrong way) means the question can never become false. The loop never ends and the tab stops responding. **Save before you run a new loop.** Then check: does the update move x *toward* false? `x = x - 40` moves it away.

What this looks like on the AP exam

REFERENCE SHEET

```
// same loop as the top of the page
REPEAT UNTIL (x > 580)
{
  circle(x, 268, 16)
  x ← x + 40
}
```

REPEAT UNTIL (…)	WHILE (…)
`x > 580`	`x <= 580`
`flakes = 150`	`flakes != 150 / flakes < 150`
`lives = 0`	`lives > 0`
`y ≥ 600`	`y < 600`

THE SLIP
The opposite of `>` is `<=`, not `<`. Same as Unit 2.

ON THE EXAM · AAP-2.K
`while` keeps going **while** true. `REPEAT UNTIL` keeps going **until** true. Same loop, **opposite** question. If the condition is already met at the start, the body runs **zero** times.

Check yourself before class

1. `let n = 0; while (n < 30) { n = n + 10; }` How many times does the body run?
2. Same loop with `n <= 30`. How many times? What is n at the end?
3. Write the `while` for `REPEAT UNTIL (y >= 600)`.
4. `let t = 50; while (t < 10) { t = t + 1; }` How many times?
5. Why can `let x = 220;` not go inside the braces?
6. Does this loop end? `let x = 0; while (x != 100) { x = x + 30; }`

Answers. 1) 3 (n: 10, 20, 30). 2) 4; n is 40. 3) `while (y < 600)`. 4) 0 — the first check is false. 5) x resets to 220 every trip; the loop never ends. 6) No: x skips 100 (90, 120…). `x < 100` is safer.

PAGE 2 — after the build. These are the patterns behind TODOs 2–5 and the asteroid lab.

Counting inside a loop — COUNTER

```
bulbs = 0;                // reset BEFORE
let x = 220;
while (x <= 580) {
  circle(x, 268, 16);
  bulbs = bulbs + 1;      // one per trip
  x = x + 40;
}
text("bulbs " + bulbs, 20, 30);
```

WHERE THE LINE GOES

`bulbs = bulbs + 1` **after** the closing brace runs once, not ten times. Reset to 0 at the top of the function, or the count grows every frame.

Every other one — LOOP + IF + %

```
if (bulbs % 2 == 0) {
  fill(230, 60, 60);     // 0, 2, 4 ... red
} else {
  fill(60, 190, 90);     // 1, 3, 5 ... green
}
circle(x, 268, 16);
```

The counter tells you **which trip** you are on. `%` from Unit 2 turns that into a pattern. `fill()` is a setting, so it must come **before** the `circle()`.

Start, question, update control different things

CHANGE	WHAT HAPPENS
start `220 → 240`	where the first one goes
question `<= → <`	where it stops (can lose the last one)
update `40 → 20`	how close together, so how many

ICICLES

Same shape of loop as the bulbs, but start at `HOUSE_LEFT + 20` and use `<`: one between every two bulbs, none on the corner. Make `let len = random(20, 50)` **inside** the loop so each icicle gets its own length.

Keep trying until it works — WHILE + RANDOM

```
ax = 400; ay = 300; tries = 0;
while (dist(ax, ay, 400, 300) < 200) {
  ax = random(800);
  ay = random(600);
  tries = tries + 1;
}
```

Here nobody knows how many trips it will take — sometimes 1, sometimes 4. That is what `while` is **for**: keep going until a condition is met. (Tomorrow's loop is for when you **do** know how many.)

AP: REPEAT UNTIL

`REPEAT UNTIL (DISTANCE ≥ 200)` — the opposite of `< 200` is `≥ 200`.

Three mistakes worth knowing about in advance

WRITTEN LIKE THIS	WHAT ACTUALLY HAPPENS
`while (x <= 580);`	The semicolon ends the loop with an empty body. x never changes — frozen tab, no error message.
`while (x <= 580) { let x = 220; … }`	x resets every trip. The question is never false. Frozen tab.
`REPEAT UNTIL (x > 580) → while (x > 580)`	Forgot to flip. x starts at 220, the first check is false, the body runs **zero** times. Nothing draws, no error.